

La bibbia delle C socket

☰ Tags	OP
# Cap	-10
⚙ Status	In progress
🕒 Ultima modifica	@November 13, 2023 12:56 AM

[UDP](#)

[TCP](#)

▼ UDP

Server

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <arpa/inet.h>
#include <unistd.h>

#define BUFFER_SIZE 1024

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <port>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    int port = atoi(argv[1]);

    // Creazione del socket
    int server_socket = socket(AF_INET, SOCK_DGRAM, 0);
    if (server_socket == -1) {
        perror("Error creating socket");
        exit(EXIT_FAILURE);
    }
}
```

```

// Configurazione dell'indirizzo del server
struct sockaddr_in server_address;
server_address.sin_family = AF_INET;
server_address.sin_addr.s_addr = INADDR_ANY;
server_address.sin_port = htons(port);

// Binding del socket all'indirizzo del server
if (bind(server_socket, (struct sockaddr *)&server_address,
        sizeof(server_address)) != 0) {
    perror("Error binding socket");
    close(server_socket);
    exit(EXIT_FAILURE);
}

printf("Server listening on port %d...\n", port);

// Ricezione dei dati dal client
char buffer[BUFFER_SIZE];
struct sockaddr_in client_address;
socklen_t client_address_len = sizeof(client_address);

while (1) {
    // Ricezione dei dati dal client
    int bytes_received = recvfrom(server_socket, buffer,
                                   sizeof(buffer), 0,
                                   (struct sockaddr *)&client_address,
                                   &client_address_len);

    if (bytes_received == -1) {
        perror("Error receiving data");
        break;
    }

    // Stampa dei dati ricevuti dal client
    buffer[bytes_received] = '\0';
    printf("Received from client: %s\n", buffer);

    // Invio della risposta al client
    const char *response = "World";
    int bytes_sent = sendto(server_socket, response, strlen(response), 0,
                           (struct sockaddr *)&client_address,
                           &client_address_len);
}

```

```

        if (bytes_sent == -1) {
            perror("Error sending data");
            break;
        }

        printf("Sent to client: %s\n", response);
    }

    // Chiusura del socket del server
    close(server_socket);

    return 0;
}

```

Client

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <arpa/inet.h>
#include <unistd.h>

#define BUFFER_SIZE 1024

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Usage: %s <server_ip> <server_port>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    const char *server_ip = argv[1];
    int server_port = atoi(argv[2]);

    // Creazione del socket
    int client_socket = socket(AF_INET, SOCK_DGRAM, 0);
    if (client_socket == -1) {
        perror("Error creating socket");
    }
}

```

```

        exit(EXIT_FAILURE);
    }

    // Configurazione dell'indirizzo del server
    struct sockaddr_in server_address;
    server_address.sin_family = AF_INET;
    server_address.sin_addr.s_addr = inet_addr(server_ip);
    server_address.sin_port = htons(server_port);

    // Invio della richiesta al server
    const char *request = "Hello";
    int bytes_sent = sendto(client_socket, request, strlen(
                                (struct sockaddr *)&server_address,
                                0, 0);
    if (bytes_sent == -1) {
        perror("Error sending data");
        close(client_socket);
        exit(EXIT_FAILURE);
    }

    printf("Sent to server: %s\n", request);

    // Ricezione della risposta dal server
    char buffer[BUFFER_SIZE];
    int bytes_received = recvfrom(client_socket, buffer, sizeof(buffer),
    if (bytes_received == -1) {
        perror("Error receiving data");
        close(client_socket);
        exit(EXIT_FAILURE);
    }

    // Stampa della risposta ricevuta dal server
    buffer[bytes_received] = '\0';
    printf("Received from server: %s\n", buffer);

    // Chiusura del socket del client
    close(client_socket);

```

```
    return 0;
}
```

▼ TCP

Server

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define MAX_BUFFER_SIZE 1024

void error(const char *msg) {
    perror(msg);
    exit(1);
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <port>\n", argv[0]);
        exit(1);
    }

    int port = atoi(argv[1]);
    if (port <= 0 || port > 65535) {
        fprintf(stderr, "Invalid port number\n");
        exit(1);
    }

    // Creazione del socket
    int server_socket = socket(AF_INET, SOCK_STREAM, 0);
    if (server_socket == -1)
        error("Error opening socket");

    struct sockaddr_in server_address;
```

```

memset(&server_address, 0, sizeof(server_address));

server_address.sin_family = AF_INET;
server_address.sin_addr.s_addr = INADDR_ANY;
server_address.sin_port = htons(port);

// Binding del socket
if (bind(server_socket, (struct sockaddr *)&server_address,
        sizeof(server_address)) != 0)
    error("Error on binding");

// Ascolto delle connessioni in ingresso
if (listen(server_socket, 5) == -1)
    error("Error on listen");

printf("Server listening on port %d...\n", port);

while (1) {
    // Accettazione di una connessione
    int client_socket = accept(server_socket, NULL, NULL);
    if (client_socket == -1)
        error("Error on accept");

    // Invio del messaggio di benvenuto al client
    const char *message = "Hello, client! This is the server\n";
    send(client_socket, message, strlen(message), 0);

    // Chiusura del socket del client
    close(client_socket);
}

// Chiusura del socket del server
close(server_socket);

return 0;
}

```

Client

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define MAX_BUFFER_SIZE 1024

void error(const char *msg) {
    perror(msg);
    exit(1);
}

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Usage: %s <server_ip> <server_port>\n", argv[0]);
        exit(1);
    }

    char *server_ip = argv[1];
    int server_port = atoi(argv[2]);
    if (server_port <= 0 || server_port > 65535) {
        fprintf(stderr, "Invalid server port number\n");
        exit(1);
    }

    // Creazione del socket
    int client_socket = socket(AF_INET, SOCK_STREAM, 0);
    if (client_socket == -1)
        error("Error opening socket");

    struct sockaddr_in server_address;
    memset(&server_address, 0, sizeof(server_address));

    server_address.sin_family = AF_INET;
    server_address.sin_port = htons(server_port);

    if (inet_pton(AF_INET, server_ip, &(server_address.sin_

```

```

        error("Invalid server IP address");

// Connessione al server
if (connect(client_socket, (struct sockaddr *)&server_a
        error("Error on connect");

// Ricezione del messaggio dal server
char buffer[MAX_BUFFER_SIZE];
memset(buffer, 0, sizeof(buffer));

if (recv(client_socket, buffer, sizeof(buffer) - 1, 0)
        error("Error on receive");

printf("Message from server: %s\n", buffer);

// Chiusura del socket del client
close(client_socket);

return 0;
}

```